Recitation 3 Worksheet: MapReduce by Hand and on Hadoop

Big Data Management · University of Haifa · Lecturer: Artem Barger

16 November 2026 · consolidate at home · not graded

How this worksheet works

In the session (Recitation 3 and Lab 3 together) the instructor took seven terms of Lecture 3 and showed each one first in `minimr.py`, a MapReduce engine in 300 lines of Python, then in the counters of a real Hadoop job on your laptop. After each term you changed one thing and observed an effect ("Your turn"). This worksheet has two parts:

- **Part A, the seven Your turns** in one table, for anyone who missed the session or wants to redo them.
- **Part B, at home**: one new question per term, answered with the *same* code and the same cluster. About 1.5 hours. It prepares Homework 1, which uses the same moves on Wikipedia data: load with `-put -` and a block size, read `fsck`, read job counters, switch a combiner off, `getmerge`.

You write what you observed into `observations.py`; `check.py` says `ok` or `WRONG` without revealing the expected value. Nothing is graded or collected. Each Part B item ends with a written question: answer it in 2–4 sentences and bring your answers to Recitation 4. Each item has hints: try for a few minutes before you read them.

Your copy of the code and the cluster

You need Python 3.9+, git, Docker with at least 6 GB of memory, and a clone of the course cluster `bigdata-management-course/hadoop-cluster` (the cluster of Lecture 3 and Homework 1; its `INSTALL.md` covers every OS). On Windows, work inside WSL2.

1. **Before the session**: `BEFORE-CLASS.md` in the clone of step 2. It builds the Hadoop image and `wordcount.jar` (both need a good connection) and ends with `python3 check.py setup`, which must print `setup: PASS`.
2. Clone the public repository `bigdata-management-course/r3-mapreduce` (no GitHub login needed) and run the checker:

   ```
   git clone https://github.com/bigdata-management-course/r3-mapreduce.git
   cd r3-mapreduce
   python3 check.py
   ```

   The last three lines are

   ```
   setup:    FAIL
   in class: c1 FAIL  c2 FAIL  c3 FAIL  c4 FAIL  c5 FAIL  c6 FAIL  c7 FAIL
   at home:  h1 FAIL  h2 FAIL  h3 FAIL  h4 FAIL  h5 FAIL  h6 FAIL  h7 FAIL
   ```

   with no traceback above them; `FAIL` only means "not done yet".
3. For the Hadoop items, start the cluster in `hadoop-cluster` with `make up put-data`, and in every terminal you use define `NN="docker exec -i hadoop-lab-namenode-1"`. `make down` stops the cluster and erases HDFS (the image and the jar stay).
4. After each item, run `python3 check.py` again and read its `PASS`/`FAIL` lines: they are your feedback. Your answers stay in your own copy; there is nothing to push or submit.

Rules and facts you need

- The data: `data/moby-dick.txt` (1,234,550 bytes), `data/pride-and-prejudice.txt` (737,950) and `data/shakespeare.txt` (5,422,627), the same three books as `input/` in HDFS. A word is a run of letters a–z after lowercasing.

- `minimr.py` prints its counters at the end of every run, then `records per reducer` and `output sha256`: equal digests mean equal answers. Hadoop prints its counters at the end of the job log, in groups (*Job Counters*, *Map-Reduce Framework*, …); save the log with `2>&1 | tee job.log`.
- In `observations.py` write numbers as numbers (`1336289`, without commas) and lists as lists. `python3 check.py c2` checks one item, `python3 check.py h1 h2` several.
- Your turn 1 edits `wordcount.py`; undo it with `git checkout wordcount.py` **before the next item**, or the later items run on changed code. `check.py` prints a `WARNING` for every shipped file that still differs.
- A Hadoop job refuses to start if its output directory exists: `$NN hdfs dfs -rm -r out-1m` first.
- Expected outputs below are exact except for timings, job ids and IP addresses. Text copied out of the PDF may lose repeated spaces; compare the words and numbers.

The tools

`python3 minimr.py JOB FILES --out DIR`	run the job module `JOB` (`wordcount`, `invindex`) on minimr
`--reducers R`	number of reducers = output files `part-r-00000` … (default 4)
`--combiner`	use the job's `combiner` at every spill and merge
`--partitioner P`	`crc32` (default), `builtin` (Python's `hash()`), or `module:function`
`--spill-records N`	map buffer size in pairs (default 250,000)
`--fail-rate p --seed s`	each map attempt crashes with probability p; `s` fixes which ones
`python3 wordcount.py map|reduce`	the same job as a Streaming program on stdin/stdout
`python3 peek.py [N]`	run map task 0 alone (buffer `N`) and list the files it leaves
`$NN hdfs dfs -D dfs.blocksize=1m -put - F < f`	write local file `f` into HDFS file `F` from stdin, 1 MB blocks
`$NN hdfs fsck /user/hadoop/D -files -blocks`	blocks of every file under `D`, their lengths, totals
`$NN hadoop jar JAR [-D ...]  IN OUT`	Java WordCount, `JAR` = `/lab/wordcount-java/target/wordcount.jar`; `-D mapreduce.job.reduces=R`, `-D wordcount.combiner=false`
`$NN hdfs dfs -getmerge OUT /tmp/f`	all `part-*` of `OUT` as one file in the container; `docker cp` it out

Part A: the seven Your turns **in class**

Slide numbers refer to `recitation03.pdf` on Moodle. For each row: make the change, run, write the value into `observations.py`, run the check, then undo the change if there is one. Rows 6 and 7 need the cluster.

Term (slides)	Change, then run	Record, check	Undo
1 map and reduce (6–10)	`wordcount.py` line 10: `yield len(word), 1`; `python3 minimr.py wordcount data/*.txt --out out-len`, then `sort -k2,2nr out-len/part-* | head -3`	`LENGTHS` (`reduce output records`), `TOP_LENGTH` (first line of `sort`); `check.py c1`	`git checkout wordcount.py`
2 shuffle and sort (12–15)	`python3 wordcount.py map < data/moby-dick.txt | python3 wordcount.py reduce | wc -l`, and the same with `| LC_ALL=C sort` between map and reduce	`LINES_UNSORTED` (the run without `sort`); `c2`	–
3 partitioner (17–20)	`--partitioner partitioners:alphabet --reducers 4 --out out-alpha`	`ALPHA_PER_REDUCER` (the list); `c3`	–
4 combiner (22–24)	`data/pride-and-prejudice.txt` alone, without and with `--combiner`	`PRIDE_SHUFFLE`, `PRIDE_SHUFFLE_COMBINED` (`reduce shuffle bytes`); `c4`	–
5 re-execution (26–29)	`--fail-rate 0.5 --seed 5`, then `--seed 23`	`FAILED_SEED5`, `GAVE_UP_MAP` (the task in `job failed:`); `c5`	–
6 HDFS blocks (31–34)	`mkdir books1m`; each book with `-D dfs.blocksize=1m -put -`; `fsck /user/hadoop/books1m -files -blocks`	`BLOCKS_1M` (`Total blocks (validated)`); `c6`	keep `books1m`
7 splits, locality (36–38)	Java WordCount on `books1m` into `out-1m`, log with `tee job-1m.log`; `getmerge`, `docker cp`, `wc -l`	`LAUNCHED_MAPS_1M`, `SHUFFLE_1M` (`Reduce shuffle bytes`); `c7`	–

Each Your-turn slide also says what you should see and why it illustrates the term. When all seven are done, the command `python3 check.py c1 c2 c3 c4 c5 c6 c7` prints `in class:  c1 PASS ... c7 PASS`.

Part B: at home

h1. Map and reduce: a second job, the inverted index — at home

Goal. Run a different job on the same engine: the inverted index of Lecture 3, Problem 1 (word → the books it appears in). Only the two functions change.

Start. `invindex.py`. Its `mapper` takes the book's name from an environment variable set by minimr and by Hadoop Streaming for each split, `mapreduce_map_input_file`, and emits `(word, book)` once per line; its `reducer` outputs the sorted list of distinct books.

Run.

```
python3 minimr.py invindex data/*.txt --out out-index
head -3 out-index/part-r-00000
```

Then count the lines whose list contains all three books (a one-line `awk` or `grep`).

Correct output. Lines like `word<TAB>["moby-dick", "shakespeare"]`: one per distinct word, so as many lines in total as word count had (`reduce output records`). The count you want is a few thousand.

Self-check. `IN_ALL_THREE`; `python3 check.py h1`.

Hints.

- One regular expression per book, e.g. starting with

```
awk -F'\t' '$2 ~ /moby-dick/ && ...' out-index/part-* | wc -l
```

- The list is JSON with the books in alphabetical order; matching three separate patterns avoids depending on that order.

Written question. Why does the mapper emit each word at most once per line (`set(...)`), and what does `invindex.combiner` do? Why may it not output the final list of books, as the reducer does?

h2. Shuffle and sort: a smaller map buffer **at home**

Goal. See that the map-side buffer only decides how many sorted runs are merged, never what the reducer receives.

Start. The demo of `peek.py` (slide 14) and the map code (slide 13, line 151): a spill happens whenever the buffer holds `--spill-records` pairs (default 250,000).

Run.

```
python3 peek.py 30000
python3 minimr.py wordcount data/*.txt --out out-small \
    --spill-records 30000
```

Correct output. `peek.py`: map task 0 now spills several times, and its four `part` files have exactly the sizes of the demo. The job: more spills than its 9 map tasks; `map output bytes`, `reduce shuffle bytes` and `output sha256` as in the term 1 demo.

Self-check. `SPILLS` = the job's `spills` counter; `python3 check.py h2`.

Hints.

- `peek.py` prints the counters of one task; the job prints them summed over all 9.
- Predict task 0's spills from its `map output records` and the buffer size before you run.

Written question. Why does the number of spills not change the output? What does it cost instead? Hadoop's counter "Spilled Records" counts records written to local disk: when would you raise the setting `mapreduce.task.io.sort.mb`?

h3. Partitioner: Hadoop with three reducers **at home**

Goal. Read the effect of a partitioner from the output files of a real job.

Start. Slide 19: the Java WordCount uses `AlphabetPartitioner` (`letter * numReducers / 26`).

Run. With the cluster up and `input/` loaded (`make up put-data`):

```
$NN hadoop jar /lab/wordcount-java/target/wordcount.jar \
    -D mapreduce.job.reduces=3 input out-3r 2>&1 | tee job-3r.log
$NN hdfs dfs -ls out-3r
$NN hdfs dfs -cat out-3r/part-r-00001 | head -1
```

Correct output. Four entries in `out-3r`: `_SUCCESS` (0 bytes) and three `part-r-0000*` files. The first is clearly the largest.

Self-check. `PARTS_3` = the three sizes in bytes as a list, in file order, e.g. `[1, 2, 3]` with your numbers; `python3 check.py h3`.

Hints.

- The size is the fifth column of `hdfs dfs -ls`.
- `Launched reduce tasks=3` in the log confirms the setting took effect.

Written question. Which first letters go to each of the three reducers? Why is the first file the largest in bytes, although the third range holds "the" and "to"? What would Hadoop's default `HashPartitioner` give instead (compare slide 19)?

h4. Combiner: how often does it run?

at home

Goal. See that the combiner may run more than once on the same data, and why that must not matter.

Start. h2 and Your turn 4: `--spill-records` and `--combiner`; slide 22 lists the places minimr calls the combiner.

Run.

```
python3 minimr.py wordcount data/*.txt --out out-sc --combiner \
    --spill-records 30000
```

Correct output. The full counter list. `combine input records` is *larger* than `map output records`; `reduce shuffle bytes` and `output sha256` equal those of the combiner demo (slide 23).

Self-check. `COMBINE_IN_SMALL` = `combine input records`; `python3 check.py h4`.

Hints.

- With several spills per task, which call site of `combine()` runs that did not run in the demo?
- What goes *into* the second round of combining: raw `(word, 1)` pairs, or already combined ones?

Written question. Explain why combine input records exceed map output records. Why is the shuffle the same as with one spill per task? What property of the function makes running it 0, 1 or 2 times safe?

h5. Re-execution: when does a job give up?

at home

Goal. Connect the attempt limit to probability: how likely is a job to fail at a given crash rate?

Start. Your turn 5: `--fail-rate 0.5` with seeds 5 and 23.

Run. Try seeds $0, 1, 2, \ldots$ until a job fails:

```
for s in 0 1 2 3 4 5 6 7 8 9; do
  echo "seed $s: $(python3 minimr.py wordcount data/*.txt --out out-s \
      --fail-rate 0.5 --seed $s 2>&1 | tail -1)"
done
```

Correct output. One line per seed; most end in `output sha256 b93105e6adb36b69`, at least one in `job failed: map ... attempt 4 crashed`.

Self-check. `FIRST_FAILING_SEED` = the smallest seed whose job fails; `python3 check.py h5`.

Hints.

- `tail -1` keeps only the last line: the digest, or the failure.
- Which map task is named can change from run to run when two tasks are on their 4th attempt; whether the job fails does not.

Written question. With 9 map tasks, crash probability $p = 0.5$ per attempt and 4 attempts, what is the probability that a job fails? Compare it with the fraction of failing seeds you saw. Why does Hadoop make `maxattempts` configurable instead of retrying forever?

h6. HDFS blocks: the last block of a file

at home

Goal. Read one file's block list and check it against the file size.

Start. Your turn 6 (`books1m`, 1 MB blocks). If you stopped the cluster since, put the books again (slide 34).

Run.

```
$NN hdfs fsck /user/hadoop/books1m/shakespeare.txt -files -blocks
```

Correct output. One line for the file (`5422627 bytes, replicated: replication=2, ... block(s): OK`), then one line per block with `len=`; all `len=` are 1,048,576 except the last.

Self-check. `LAST_BLOCK_1M` = the `len=` of the last block; `python3 check.py h6`.

Hints.

- Blocks are numbered from 0.
- Check your number with the file size and the block size before you record it.

Written question. Does the last block take 1 MB of disk on the datanodes? How many map tasks does WordCount launch for this one file, and which of them reads the fewest bytes? What happens to a line that starts in one block and ends in the next?

h7. YARN containers: bigger maps, fewer at once

at home

Goal. See YARN's capacity as a fit problem: container size decides how many tasks run at the same time.

Start. Slide 36: the NodeManager offers 2048 MB; the AM and every task get 512 MB. Your turn 7's log `job-1m.log` is the 512 MB baseline.

Run.

```
$NN hadoop jar /lab/wordcount-java/target/wordcount.jar -D mapreduce.job.reduces=2 \
    -D mapreduce.map.memory.mb=1024 -D mapreduce.map.java.opts=-Xmx768m \
    books1m out-mem 2>&1 | tee job-mem.log
grep -E "map [0-9]+% |time spent by all map" job-mem.log
```

Correct output. The progress lines (`map 11% ...`) now arrive several seconds apart instead of about one second; the counters `Total time spent by all maps in occupied slots (ms)` and `Total time spent by all map tasks (ms)`.

Self-check. `SLOT_RATIO` = the first counter divided by the second, as a whole number; `python3 check.py h7`.

Hints.

- Slide 37 does the same division for the 512 MB run.
- `-Xmx768m` keeps the Java heap inside the larger container; without it the task still uses 384 MB.

Written question. What unit is a "slot" (look in `conf/yarn-site.xml` of `hadoop-cluster`), and why does the ratio follow from the container size? How many map containers fit next to the AM in 2048 MB now, and how does that explain the slower progress lines? Compare the time from `map 0%` to `map 100%` in both logs. When would you *want* bigger containers despite this?

Further reading: J. Dean & S. Ghemawat, "MapReduce: Simplified Data Processing on Large Clusters", OSDI 2004; T. White, *Hadoop: The Definitive Guide*, 4th ed. (O'Reilly, 2015), ch. 2 (MapReduce), 3 (HDFS), 4 (YARN), 7 (how MapReduce works); J. Lin & C. Dyer, *Data-Intensive Text Processing with MapReduce* (2010), ch. 2–3. The Lab 3 deck (`lab03.pdf`) and the self-study slides after the closing slide of `recitation03.pdf` go deeper into every term.